

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto "AtomReactChamber"

Curso: *Base de Datos II*

Docente: Ing. Patrick Cuadros Quiroga

Integrante:

Laos González, Adriana Rafaela (2023077474)

Tacna – Perú
2026

Sistema “AtomReactChamber”
Informe Final

Versión *1.0*

CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	Adriana Laos	Ing. Patrick Cuadros Quiroga (pendiente)	(pendiente)	2026-11-07	Informe final de la versión 2.0, antes del piloto

ÍNDICE GENERAL

1. Resumen ejecutivo.....4

2. Objetivos vs. resultados.....4

3. Alcance logrado vs. planeado.....6

4. Arquitectura final.....6

5. Base de datos.....7

6. Automatizaciones.....8

7. Integraciones.....8

8. Pruebas y calidad.....9

9. Gestión del proyecto.....9

9.1 Cronograma.....9

9.2 Contribuciones.....10

10. Costos.....10

11. Lecciones aprendidas.....10

12. Limitaciones.....10

13. Trabajo futuro.....11

14. Conclusiones.....11

15. Recomendaciones.....12

16. Anexos.....12

1. Resumen ejecutivo

AtomReactChamber: laboratorio virtual gamificado de química para estudiantes de secundaria del Perú es una aplicación web que convierte la tabla periódica en un mapa de misiones. Cada casilla es una práctica guiada cuyos pasos se verifican con un simulador molecular 2D; el asistente **Quimi** da ayudas escalonadas que cuestan XP; la base de datos calcula estrellas, XP, rachas e insignias; y el docente sigue el avance de su aula en un tablero.

La versión 2.0, desarrollada por una sola integrante en el curso **Base de Datos II** (UPT – FAING – EPIS), reemplaza a la v0.1 (un simulador en un único componente React, sin base de datos, usuarios ni pruebas). El resultado entregado es:

- **Base de datos PostgreSQL 16** en 3FN gestionada con Liquibase: 29 tablas, 6 vistas, 1 vista materializada, 16 funciones (9 de negocio y 7 de *trigger*), 10 *triggers* y 73 índices, en 48 *changesets* con *rollback*.
- **API REST** Fastify 5 + TypeScript con JWT, roles, OpenAPI y 17 pruebas de integración contra PostgreSQL real.
- **Frontend** React 18 + Vite + TypeScript con 8 páginas, modo demostración sin servidor y verificación de accesibilidad WCAG 2.2 AA con axe.
- **9 automatizaciones** de GitHub Actions (infraestructura, base de datos, respaldos, *release*, documentación, GitHub Pages, CI, CodeQL y monitoreo de disponibilidad) y **Terraform** modular para Neon y Render.
- **Integraciones:** extensión de VS Code, servidor MCP, *skill* y subagente de Claude Code.
- **70 pruebas automatizadas** y **S/ 0** de costo mensual de infraestructura.

Los objetivos que dependen de estudiantes reales (tasa de finalización, aprendizaje, usabilidad SUS) quedan **pendientes del piloto**: los instrumentos de medición ya están implementados en la base de datos.

2. Objetivos vs. resultados

Objetivo general (FD01 §1.4.1): desarrollar e implementar AtomReactChamber con base de datos PostgreSQL, logrando en un piloto una tasa de niveles completados ≥ 60 % y SUS ≥ 70. **Estado:** producto desarrollado e implementado; las dos metas del piloto están pendientes.

ID	Objetivo	Indicador	Meta	Resultado actual	Evidencia	Estado
OE-1	BD en 3FN con migraciones versionadas	*Changesets* con *rollback* probado	100 % de objetos versionados	48 *changesets* en 13 archivos; todos con `--rollback`; `update-testing-rollback` en BD limpia	`db/changelog/`, `.github/workflows/db.yml`, `docs/database/diccionario-de-datos.md`	✔️ Cumplido
OE-2 / OS-1	Catálogo de misiones alineadas al CNEB	Niveles publicados y validados	9 niveles en 4 temas (meta futura ≥ 12)	9 niveles en 4 misiones; un *solver* demuestra que los 9 son completables	`db/data/juego.json`, `frontend/src/engine/engine.test.ts`	✔️ Cumplido (validación por docente de química: pendiente)
OE-3 /	Finalización de misiones	Completados / iniciados	≥ 60 %	Instrumento listo (`v_estadistic`	`db/changelog/`	◖ Pendiente de piloto

ID	Objetivo	Indicador	Meta	Resultado actual	Evidencia	Estado
OS-2				`as_nivel.tasa_completado`, KPI del tablero); solo hay datos de demostración	`sql/009-vistas.sql`, página `Dashboard`	
OE-4 / OS-3	Desempeño en el cuestionario	% de aciertos 1.er vs. último intento	+15 p. p.	Respuestas corregidas por la BD en `respuesta`; vista `v_preguntas_dificiles`	`005-progreso.sql`, `008-triggers.sql`	◖ Pendiente de piloto
OS-4	Duración de pasos	Mediana de tiempo por paso	≤ 3 min	`intento_paso.segundos` y `v_tiempo_por_paso.segundos_mediana`	`009-vistas.sql`	◖ Pendiente de piloto
OE-5 / OS-5	Quimi orienta sin resolver	% de pasos con ayuda “solución”	≤ 30 %	`pista_usada` ⋈ `pista` (`nivel_ayuda = 3`)	`fn_usar_pista`	◖ Pendiente de piloto
OE-6 / OS-6	Accesibilidad	Lighthouse ≥ 90; violaciones WCAG	≥ 90 en 5 vistas	axe-core: **0 violaciones graves o críticas** WCAG 2.2 A/AA en 4 rutas; Lighthouse aún no automatizado	`frontend/e2e/app.spec.ts`	◖ Parcial
OI-3	Usabilidad percibida	SUS promedio	≥ 70	Cuestionario SUS preparado para el piloto	—	◖ Pendiente de piloto
OE-7 / OS-7	Disponibilidad	Chequeos `/api/health` OK / total	≥ 99 % (07:00–18:00)	`salud.yml` consulta cada 6 h y deja el resultado en el resumen de Actions; el indicador se calcula al acumular ejecuciones en producción	`.github/workflows/salud.yml`, `release.yml` (*smoke test*)	◖ Instrumento listo
OE-8 / OS-8	Calidad del software	Cobertura backend; workflows en verde	≥ 70 %; 100 %	70 pruebas automatizadas (17 + 15 + 7 + 19 + 12); los *workflows* terminan en verde aunque falten *secrets*; reporte de cobertura aún no configurado	`backend/test`, `frontend/src/engine`, `frontend/e2e`, `extensions/vscode/test`, `integrations/mcp/test`	◖ Parcial
OS-9	Gestión docente	Crear aula y asignar misión en ≤ 5 min	100 % de docentes	Crear aula y unirse con código: implementado y probado (PI-16); la asignación se hace por API (formulario pendiente)	`AulaService`, página `Aula`	◖ Pendiente de piloto

ID	Objetivo	Indicador	Meta	Resultado actual	Evidencia	Estado
OE-9 / OS-10	Costo de operación	Costo mensual de infraestructura	S/ 0	**S/ 0**: Neon Free, Render Free, GitHub Pages, HCP Terraform Free; referencia AWS/Azure ≈ USD 38/mes	`infra/README.md` §6, `infra.yml` (Infracost)	✔️ Cumplido
OI-1	Brecha de laboratorios (ESCALE)	% de locales con laboratorio	Cifra reproducible	Sin avance (tarea de investigación)	`docs/investigacion/problema-y-objetivos.md`	✖️ Pendiente
OI-2	Ganancia de aprendizaje	⟨g⟩ de Hake	≥ 0,30	Se medirá con un pre-test/post-test aplicado fuera de la aplicación	—	◖ Pendiente de piloto
OI-4	Pistas escalonadas	% de pasos completados sin “solución”	≥ 70 %	Consulta sobre `pista_usada` e `intento_paso` disponible	—	◖ Pendiente de piloto

3. Alcance logrado vs. planeado

N.° (FD02)	Característica	Planeado	Logrado	Observación
001	Aplicación web cliente-servidor	✔️	✔️	React + Fastify + PostgreSQL
002	Mapa de misiones con forma de tabla periódica	✔️	✔️	Componente `TablaPeriodica`
003	Niveles guiados verificados por el simulador	✔️	✔️	9 niveles, DSL de condiciones
004	Quimi: ayudas escalonadas + glosario FTS	✔️	✔️	87 ayudas, 39 términos
005	Gamificación (XP, estrellas, insignias, rachas, ranking)	✔️	✔️	Calculada en la BD; 8 insignias
006	Cuestionario de cierre con retroalimentación	✔️	✔️	27 preguntas, corrección por *trigger*
007	Roles, aulas con código, asignación, progreso	✔️	◖	Asignación de niveles solo por API
008	Tablero de uso	✔️	✔️	Vista materializada `mv_uso_diario`
009	Laboratorio libre	✔️	✔️	4 experimentos predefinidos
010	Integraciones (VS Code, Claude Code, MCP, GitHub)	✔️	✔️	`docs/integraciones.md`
—	Modo demostración sin servidor	✔️	✔️	`FuenteDemo`; publicado en GitHub Pages
011–017	3D, PDF por correo, números cuánticos, editor de prácticas, LLM, ≥ 12 niveles, consentimiento parental	Trabajo futuro	—	Sección 13

4. Arquitectura final

Arquitectura en capas cliente-servidor (detalle en el **FD04 – Documento de Arquitectura de Software**):

- **Navegador:** SPA React con el motor de simulación en TypeScript puro (`engine/`), dibujo en Canvas 2D (`render/dibujar.ts`) y dos fuentes de datos intercambiables (`FuenteApi`, `FuenteDemo`).
- **API Fastify** en Render (plan Free): rutas → servicios/repositorios → `Database` (pool `pg` de 5 conexiones).
- **PostgreSQL 16** en Neon (plan Free): reglas de negocio en funciones y *triggers*.
- **GitHub:** Actions (CI/CD), Pages (demo, documentación, TypeDoc, diccionario, reportes) y Releases.

Decisiones clave: un solo lenguaje (TypeScript); reglas de negocio en la base de datos; *offline-first* frente al arranque en frío del plan gratuito; Quimi basado en reglas (sin LLM) por seguridad con menores y costo cero.

5. Base de datos

Objeto	Cantidad	Ejemplos
Tablas	29	`usuario`, `nivel`, `paso`, `intento`, `pista_usada`, `respuesta`, `evento`
Vistas	6	`v_progreso_estudiante`, `v_ranking_aula`, `v_estadisticas_nivel`
Vista materializada	1	`mv_uso_diario`
Funciones de negocio	9	`fn_finalizar_intento`, `fn_usar_pista`, `fn_quimi_buscar`
Funciones de *trigger*	7	`fn_auditar`, `fn_tg_respuesta_validar`
Triggers	10	`tg_respuesta_validar`, `tg_aula_docente`
Índices	73 (30 declarados con `CREATE INDEX`)	GIN (FTS), BRIN (telemetría), parciales, de expresión
Changesets Liquibase	48 en 13 archivos	`001-extensiones` … `010-indices-fk`, semillas `900–902`

Decisiones de diseño principales:

1. **Reglas de negocio en PostgreSQL:** estrellas, XP (25 % en repeticiones), ayudas escalonadas, racha (días con al menos un nivel completado), insignias y corrección del cuestionario se aplican igual para todo cliente.
2. **3FN con desnormalización controlada:** ΔH de reacción y ranking se calculan en vistas; los contadores de `intento` y la XP del usuario solo los modifican funciones y *triggers*.
3. **Índices con propósito:** GIN para el glosario de Quimi, BRIN para tablas de telemetría ordenadas en el tiempo, índices parciales (`ux_opcion_una_correcta`, `ix_intento_completado`) y el *changeset* 010 con los 11 índices de claves foráneas detectados con tbls.
4. **Contenido como datos:** `db/data/*.json` es la única fuente; un generador produce las semillas SQL y el contenido del modo demo.
5. **Auditoría y respaldo:** `fn_auditar` registra cambios sin contraseñas; `backup.yml` hace `pg_dump` diario cifrado y verifica la restauración comparando filas.

6. Automatizaciones

Fila de la rúbrica	Workflow	Evidencia
Infraestructura (Terraform) + pruebas + costos	`infra.yml`	Resumen de calidad IaC (fmt, validate, tflint, checkov), *Reporte de pruebas superadas* (JUnit), reporte de costos (USD 0 frente a AWS/Azure con Infracost)
Base de datos (Liquibase)	`db.yml`	`validate`, `update-sql`, `update`, prueba de *rollback*, inventario de objetos; artefacto `reporte-liquibase-*`
Copias de seguridad	`backup.yml`	`pg_dump` diario, SHA-256, cifrado AES-256, restauración verificada; retención abuelo-padre-hijo
Release y despliegue	`release.yml`	release-please (versión semántica y CHANGELOG), artefactos `.zip`/`.vsix`, Liquibase → *deploy hooks* → *smoke test*
Documentación en README	`docs.yml`	tbls (ER + diccionario), diagramas Mermaid, secciones generadas del README
Documentación técnica	`pages.yml`	Sitio con TypeDoc, BD, diagramas, reportes y demo `/app/`
Integración continua	`ci.yml`	*Lint*, *typecheck*, actionlint, pruebas
Seguridad del código	`codeql.yml`, `dependabot.yml`	Alertas CodeQL; PR de dependencias
Disponibilidad	`salud.yml`	`GET /api/health` cada 6 h

Todos los *workflows* terminan en verde aunque falten los *secrets* de la nube: los pasos de despliegue se omiten y el reporte lo indica. Detalle en `docs/automatizaciones.md`.

7. Integraciones

Integración	Ubicación	Qué hace	Pruebas
Extensión de VS Code	`extensions/vscode`	Vistas de misiones y catálogo, preguntar a Quimi, explicar reacción con perfil de energía, inicio de sesión (token en `SecretStorage`), barra de estado	19
Servidor MCP	`integrations/mcp`	Herramientas `buscar_especie`, `explicar_reaccion`, `listar_niveles`, `preguntar_quimi`, `resumen_dashboard` y recurso `arc://catalogo`	12
Skill + subagente de Claude Code	`.claude/skills/atomreactchamber`, `.claude/agents/tutor-quimi.md`	Consultas a la API con `curl`; tutor que explica sin revelar respuestas	—
GitHub Actions / Pages / Releases	`.github/workflows`	Construye, prueba, publica el `.vsix` y la documentación	—

Cada integración registra su uso con un `origen` propio (`vscode`, `mcp`, `claude`) y el tablero lo muestra. Detalle en `docs/integraciones.md`.

8. Pruebas y calidad

Suite	Herramienta	Pruebas	Qué verifica
Integración del backend	Vitest + `app.inject` + PostgreSQL 16	17	Cada *endpoint* y cada regla de la BD (orden de pasos, ayudas, estrellas, XP, 25 %, insignias, desbloqueo, aulas, telemetría)
Motor de simulación	Vitest	15	Química (ΔH, Clausius-Clapeyron, fases, fotones), DSL de condiciones y un *solver* que completa los 9 niveles
Extremo a extremo	Playwright + axe-core	7	Inicio, nivel 1 completo con estrellas, dashboard docente y WCAG 2.2 AA en 4 rutas
Extensión VS Code	Vitest	19	Formato, perfil de energía, vistas
Servidor MCP	Vitest	12	Herramientas y recurso
Total		**70**	

Accesibilidad: axe-core no reporta violaciones graves ni críticas de WCAG 2.0, 2.1 y 2.2 (A y AA) en `/`, `/#/mapa`, `/#/laboratorio` e `/#/ingresar`. La interfaz usa HTML semántico, navegación por teclado, anuncios `aria-live` del progreso, tipografía Atkinson Hyperlegible y respeta `prefers-reduced-motion`.

Seguridad: bcrypt, JWT, roles y propiedad, consultas parametrizadas, Helmet, CORS, límite de solicitudes, CodeQL y Dependabot (FD03 RNF-04 a RNF-06).

9. Gestión del proyecto

9.1 Cronograma

Fase (FD01 §1.2)	Semanas planeadas	Estado	Evidencia (commits)
Análisis y BD	S1–S4	Terminada	`feat(db): modelo de datos PostgreSQL con Liquibase…`; `feat(db): índices para claves foráneas…`
API	S5–S8	Terminada	`feat(api): API REST Fastify + TypeScript con JWT, roles y OpenAPI`
Frontend y simulador	S6–S12	Terminada	`feat(frontend): AtomReactChamber v2 gamificado…`; `fix(contenido): calibrar nivel 5…`
Automatizaciones e integraciones	S10–S14	Terminada	`ci: automatizaciones GitHub Actions e infraestructura Terraform`; `feat(integraciones): …`
Piloto y documentación	S12–S16	Documentación terminada; **piloto pendiente**	`docs: …` (FD01–FD05, manuales)

La versión v0.1 data del 2026-10-02; la integración de la v2 en el repositorio se registró el 2026-11-07. El historial completo está en `git log`.

9.2 Contribuciones

Integrante	Roles	Contribución
Laos González, Adriana Rafaela (2023077474)	Gestora, analista, diseñadora de BD, desarrolladora backend y frontend, DevOps, QA y documentadora	**100 %**

Se usaron asistentes de IA (Claude Code y sus *skills*) como herramienta de apoyo, bajo dirección y revisión de la integrante; las decisiones de alcance y de diseño constan en `task_plan.md`.

10. Costos

Concepto	Planeado (FD01)	Real
Infraestructura mensual (Neon, Render, GitHub Pages, HCP Terraform)	S/ 0,00	**S/ 0,00** (planes gratuitos)
Costos generales (depreciación de laptop)	S/ 316,67	Igual (imputado)
Costos operativos (electricidad, internet)	S/ 97,31	Igual (estimado)
Personal (256 h × S/ 15)	S/ 3 840,00	Igual (costo de oportunidad estimado)
Total del proyecto	**S/ 4 253,98**	**S/ 4 253,98**

Una operación en nube de pago (referencia AWS o Azure) costaría alrededor de USD 38 mensuales según la arquitectura de referencia estimada con Infracost (`infra/terraform/cost-reference/`).

11. Lecciones aprendidas

1. **Poner las reglas en la BD simplificó los clientes:** la web, la extensión y el servidor MCP comparten las mismas reglas sin duplicar código; el costo es que las pruebas necesitan un PostgreSQL real.
2. **Liquibase elimina los comentarios del cuerpo de las funciones:** la documentación debe ir en `COMMENT ON`.
3. **El plan gratuito obliga a diseñar para el arranque en frío:** el modo demo resultó clave para exponer y para aulas con conectividad irregular.
4. **Un *solver* automático del contenido** detectó un nivel mal calibrado (combustión del carbono) antes de que lo encontrara un estudiante.
5. **Generar el diccionario desde la BD** evita que la documentación quede desactualizada y reveló 11 claves foráneas sin índice.
6. **Corregir los documentos contra el código** encontró nombres de objetos inexistentes en los objetivos; los instrumentos de medición deben citar objetos reales.

12. Limitaciones

⚠️ **NOTA IMPORTANTE — Consentimiento de padres (trabajo futuro).** La versión actual no incluye el flujo de consentimiento de quienes ejercen la patria potestad para estudiantes menores de 14 años (Ley 29733 y D. S. 016-2024-JUS). Hasta implementarlo, la IE debe recoger ese consentimiento fuera de la aplicación antes de que estudiantes de 1.º y 2.º de secundaria creen su cuenta.

- ⚠️ **Consentimiento de padres (Ley 29733, D. S. 016-2024-JUS):** la aplicación **no implementa** el flujo de consentimiento para estudiantes menores de 14 años. Hasta implementarlo, el consentimiento debe recogerse fuera de la aplicación (formulario de la IE) antes de cualquier piloto con 1.º y 2.º de secundaria.
- **Sin datos de estudiantes reales:** las métricas del tablero provienen de datos de demostración; los objetivos del piloto no están medidos.
- **9 niveles:** la meta de ≥ 12 niveles quedó como meta futura.
- **Asignación de niveles** disponible solo por API; falta el formulario en la página `Aula`.
- **JWT de 7 días:** `usuario.activo` se verifica solo al iniciar sesión; una cuenta desactivada conserva su token hasta que expire.
- **Verificación de pasos en el cliente:** un usuario técnico podría marcar pasos sin cumplirlos (se acepta por ser una herramienta formativa).
- **Arranque en frío** del plan gratuito de Render (hasta ~1 min).
- **Energías de activación pedagógicas**, no reales (declaradas en la interfaz).
- Sin Lighthouse CI ni reporte de cobertura todavía.

13. Trabajo futuro

1. Laboratorio en 3D.
2. Quimi con modelo de lenguaje (LLM) en la 3.ª unidad, con controles de seguridad para menores, manteniendo la interfaz de `QuimiService`.
3. Reportes PDF enviados por correo al docente.
4. Módulo de números cuánticos.
5. Editor de prácticas para docentes (hoy el contenido se edita en `db/data`).
6. **Flujo de consentimiento de padres** para menores de 14 años.
7. **Pre-test/post-test** para medir la ganancia de aprendizaje (OI-2), primero fuera de la aplicación y luego con un *changeset* que distinga pre y post.
8. Ampliar el catálogo a ≥ 12 niveles.
9. Formulario de asignación de niveles, *refresh tokens*, Lighthouse CI y cobertura.

14. Conclusiones

1. AtomReactChamber v2 cumple el alcance funcional planeado en el FD02: mapa de misiones, niveles guiados verificados por simulador, Quimi, gamificación, cuestionarios, aulas, tablero, laboratorio libre, modo demo e integraciones.
2. La base de datos es el núcleo del producto: concentra las reglas de negocio, la integridad y las métricas de los objetivos, con migraciones versionadas, respaldos verificados y documentación generada.
3. La calidad está respaldada por 70 pruebas automatizadas, verificación de accesibilidad WCAG 2.2 AA y nueve *workflows* que funcionan sin costo.
4. La validación educativa (finalización, aprendizaje y usabilidad) depende del piloto, para el que los instrumentos ya están listos; antes debe resolverse el consentimiento de padres para menores de 14 años.

15. Recomendaciones

1. Ejecutar el piloto con al menos un aula (n ≥ 30), con consentimiento de padres gestionado por la IE, pre-test y post-test, y el cuestionario SUS.
2. Validar los 9 niveles con un docente de química antes del piloto.
3. Configurar los *secrets* de producción para activar despliegue, respaldos cifrados y monitoreo.
4. Priorizar en la siguiente iteración el consentimiento parental y el formulario de asignación de niveles.

16. Anexos

Documento	Ubicación
FD01 Informe de Factibilidad	`docs/fd/FD01-Informe-Factibilidad.md`
FD02 Documento de Visión	`docs/fd/FD02-Informe-Vision.md`
FD03 Especificación de Requerimientos (SRS)	`docs/fd/FD03-Informe-SRS.md`
FD04 Documento de Arquitectura (SAD)	`docs/fd/FD04-Informe-SAD.md`
Manual de usuario	`docs/manuales/manual-de-usuario.md`
Manual técnico	`docs/manuales/manual-tecnico.md`
Diccionario de datos	`docs/database/diccionario-de-datos.md`
Estándar de programación	`docs/estandares/estandar-de-programacion.md`
Automatizaciones	`docs/automatizaciones.md`
Integraciones	`docs/integraciones.md`
Problema y objetivos	`docs/investigacion/problema-y-objetivos.md`
Infraestructura	`infra/README.md`